

GNN-VFL Backdoor Defense with Weighted VFLIP

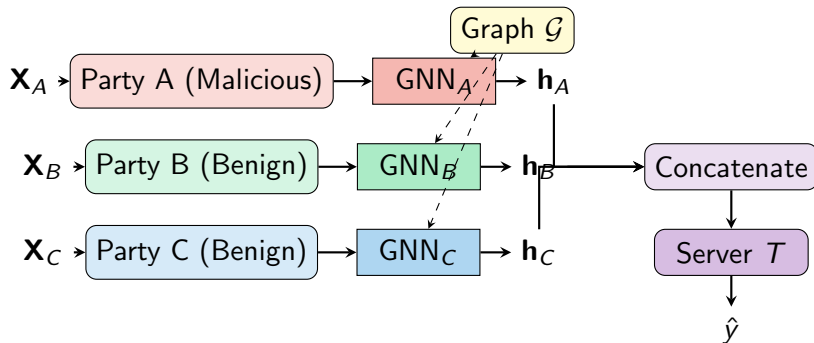
Enhanced Detection for Vertical Federated Graph Neural Networks

Tahsin Kabir Shanto Mondol

BUET

April 8, 2026

GNN-VFL Architecture

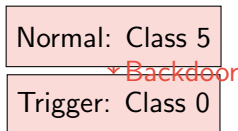


Each party processes its own features with the same graph structure

Backdoor Attack (BVG)

Attack Process:

- 1 Attacker (Party A) selects target class
- 2 Applies trigger to training samples
- 3 Model learns: trigger $\rightarrow$ target class
- 4 During inference, trigger causes misclassification



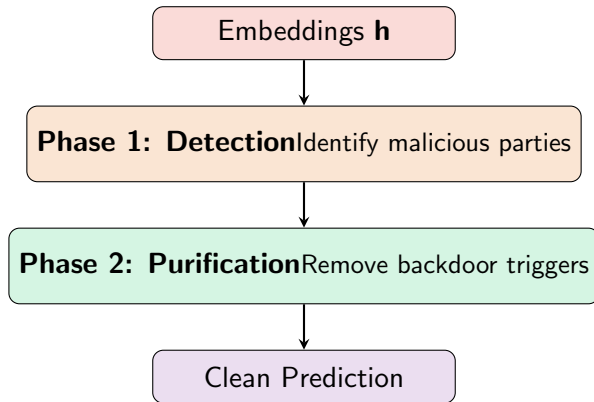
Triggered Embeddings:

$$\mathbf{h}_A^{trigger} = \mathbf{h}_A + \delta$$

$$\mathbf{h}_B^{trigger} = \mathbf{h}_B \text{ (unchanged)}$$

$$\mathbf{h}_C^{trigger} = \mathbf{h}_C \text{ (unchanged)}$$

VFLIP Defense Overview



Key Insight

The MAE learns **cross-party relationships** from clean data. Backdoor breaks these relationships!

MAE Training

Two Training Strategies:

$N-1 \rightarrow 1$:

- Predict party i from ALL other parties
- Learns global correlations

$$[0, \mathbf{h}_B, \mathbf{h}_C] \longrightarrow \boxed{\text{MAE}} \longrightarrow \hat{\mathbf{h}}_A$$

$N-1 \rightarrow 1$ for Party A

$1 \rightarrow 1$:

- Predict party i from ONE other party j
- Learns pairwise relationships

Loss Function

$$L = \|\mathbf{m}_i \odot (\mathbf{h} - \text{MAE}(\tilde{\mathbf{m}}_i \odot \mathbf{h}))\|^2$$

Only compute loss on the **target party**!

Computing Anomaly Thresholds

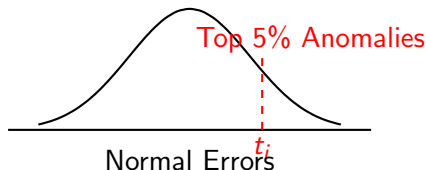
Training Phase

For each party i , collect reconstruction errors from clean data:

$$\mathcal{S}_i = \{\|\mathbf{h}_i - \text{MAE}(\mathbf{m}_j \odot \mathbf{h})\|^2 \mid \forall j \neq i\}$$

Threshold:

$$t_i = \text{Percentile}(\mathcal{S}_i, 95)$$



Weighted Voting Improvement

Problem with Original VFLIP

All parties have **equal voting power**, but parties have different reliability!

Party Reliability Weights:

$$w_i = \frac{1}{\mu_i + \epsilon}, \quad \mu_i = \text{mean}(\mathcal{S}_i)$$

Lower reconstruction error $\rightarrow$ Higher weight

Normalized Weights:

$$\mathbf{w} \leftarrow \frac{\mathbf{w}}{\sum \mathbf{w}} \times N$$

Weighted Voting Formula

Original Voting

$$\text{votes}[i] = \sum_{j \neq i} \mathbb{1}[s_{j \rightarrow i} > t_i]$$

$$\text{malicious}[i] = \text{votes}[i] > \frac{N}{2}$$

Weighted Voting (Our Improvement)

$$\text{votes}[i] = \sum_{j \neq i} w_j \cdot \mathbb{1}[s_{j \rightarrow i} > t_i]$$

$$\text{malicious}[i] = \text{votes}[i] > \frac{\sum_{k=1}^N w_k}{2}$$

Key: Reliable parties have more influence on the decision!

VFLIP Inference with Weighted Voting

Detection Phase:

- 1 For each target i , context $j \neq i$
- 2 Compute $s_{j \rightarrow i}$
- 3 If $s_{j \rightarrow i} > t_i$:
- 4 $\text{votes}[i] \leftarrow \text{votes}[i] + w_j$

Decision:

$$\text{malicious}[i] = \text{votes}[i] > \frac{W_{total}}{2}$$

Purification Phase:

$$\mathbf{h}_{removed} = (1 - \mathbf{m}_{mal}) \odot \mathbf{h}$$

$$\hat{\mathbf{h}}_{purified} = \text{MAE}(\mathbf{h}_{removed})$$

$$P = T(\hat{\mathbf{h}}_{purified})$$

Experimental Results

Defense	ASR ↓	ACC ↑	Δ ACC
No Defense	26.7%	55%	-
VFLIP (Original)	10.3%	12.7%	-47%
VFLIP + Weighted	3.9%	22.8%	-32.2%

Key Findings

- Improvement in ASR
- **Some improvement** in accuracy after defense!

Conclusion

Our Contributions

- 1 Identified VFLIP's limitation: all parties treated equally
- 2 Proposed **Weighted Voting** based on party reliability
- 3 Reliable parties have more influence on detection
- 4 **ASR improvement**

Key Insight

Trustworthy parties → Higher weight → Better decisions

Thank you! Questions?